
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 14

Outros Tópicos sobre C

Exercício de Autorrevisão (14.1)

19 lacunas conceituais

Abordagem incremental: Capítulos 1 a 14

O capítulo final do núcleo de C

Construindo sobre os Capítulos Anteriores

O Capítulo 14 – “Outros Tópicos sobre C” – amarra os últimos fios da linguagem, completando seu conjunto de ferramentas para programas profissionais em C. Construindo sobre os Caps. 1–13, agora incorporamos:

- **Redirecionamento de E/S** – `<`, `>`, `»`, `|` (pipe) do shell
- **Argumentos variádicos** – funções como `printf` que aceitam número variável de argumentos via `<stdarg.h>`: `va_list`, `va_start`, `va_arg`, `va_end`
- **Argumentos de linha de comando** – `int main(int argc, char *argv[])`
- **Compilação multi-arquivo** – `extern`, vinculação, ferramenta `make` e `Makefile`
- **Término de programa** – `exit`, `atexit`, `abort`
- **Sufixos numéricos** – `L`, `U`, `F` para controlar tipo de constantes
- **Tratamento de sinais** – `<signal.h>`, `signal`, `raise`, `SIGINT`, `SIGABRT`, ...
- **Alocação dinâmica avançada** – `calloc` (com zeros), `realloc` (redimensiona)
- **Arquivos temporários** – `tmpfile`
- **Qualificadores adicionais** – `const`, `volatile`
- `goto` – desvio incondicional (uso restrito)

Boa prática de programação

Este é o capítulo que separa programadores casuais de programadores profissionais. Argumentos de linha de comando habilitam ferramentas Unix-style; `Makefile` viabiliza projetos grandes; `calloc`/`realloc` são essenciais para estruturas dinâmicas robustas; `signal` permite programas resilientes.

Contents

1 Exercício 14.1 – Preenchimento de Lacunas (19 itens)

2

1. Exercício 14.1 – Preenchimento de Lacunas (19 itens)

Resposta detalhada

- a) Símbolo que redireciona entrada de arquivo (em vez do teclado): < (redirecionamento de entrada).

```
1 ./meuprog < entrada.txt
```

- b) Símbolo que redireciona saída para arquivo (em vez da tela): > (redirecionamento de saída).

```
1 ./meuprog > saida.txt
```

- c) Símbolo que acrescenta saída ao final de um arquivo: » (acréscimo de saída).

```
1 ./meuprog >> log.txt
```

Diferença crucial: > *trunca* o arquivo; » *preserva* o conteúdo existente e acrescenta.

- d) Direciona saída de um programa para entrada de outro: | (pipe).

```
1 ls | sort | uniq
```

A saída de `ls` torna-se entrada de `sort`, cuja saída torna-se entrada de `uniq`. Essa composição é a essência da filosofia Unix.

- e) Lista de parâmetros indicando número variável de argumentos: ... (reticências).

```
1 int media( int contagem, ... );
```

`printf` e `scanf` são variádicas.

- f) Macro chamada *antes* de acessar argumentos variáveis: `va_start`.

```
1 va_list args;
2 va_start( args, contagem ); /* contagem = ultimo
   arg nomeado */
```

- g) Macro que acessa argumentos individuais da lista variável: `va_arg`.

```
1 int valor = va_arg( args, int ); /* le proximo
   argumento */
```

- h) Macro que finaliza o processamento da lista variável: `va_end`.

```
1 va_end( args ); /* SEMPRE chamar antes do return!
   */
```

- i) Argumento de `main` que recebe *quantidade* de argumentos da linha de comando: `argc` (*argument count*).

- j) Argumento de `main` que armazena os argumentos como strings: `argv` (*argument vector*).

```
1 int main( int argc, char *argv[] )
2 {
```

```

3      /* argv[0] = nome do programa
4         argv[1], argv[2], ... = argumentos */
5  }

```

- k) Utilitário do Linux/Unix que processa arquivo de regras: **make**, que lê um Makefile.

```

1  # Exemplo de Makefile
2  programa: main.o util.o
3      gcc -o programa main.o util.o
4
5  main.o: main.c util.h
6      gcc -c main.c

```

- l) Função que força término da execução: **exit**.

```

1  exit( 0 );           /* terminacao bem-sucedida */
2  exit( EXIT_FAILURE ); /* terminacao com erro */

```

- m) Função que registra rotina a ser chamada no término normal: **atexit**.

```

1  void cleanup( void ) { printf( "Limpendo...\n" ); }
2  int main( void ) {
3      atexit( cleanup ); /* chamada automatica ao
4                          final */
5      return 0;
6  }

```

- n) Letra anexada a constantes para especificar tipo exato: **sufixo**.

```

1  3.14F      /* float (sem F seria double) */
2  100UL      /* unsigned long */
3  99999999L  /* long int */
4  1.5L       /* long double */

```

- o) Função que abre arquivo temporário (existe até fechar ou fim do programa): **tmpfile**.

```

1  FILE *fp = tmpfile(); /* arquivo binario,
                          automaticamente removido */

```

- p) Função para interceptar eventos inesperados: **signal**.

```

1  void tratador( int sig ) { /* trata o sinal */ }
2  signal( SIGINT, tratador ); /* registra handler
                               para Ctrl+C */

```

- q) Função que gera sinal de dentro do programa: **raise**.

```

1  raise( SIGABRT ); /* gera SIGABRT explicitamente
                     */

```

- r) Função que aloca array e *inicializa em zero*: **calloc**.

```

1  int *arr = calloc( 100, sizeof( int ) );
2  /* 100 inteiros, todos zero */

```

Diferença de malloc: calloc aceita 2 argumentos (n e tamanho) e garante zeros; malloc deixa memória com lixo.

s) Função que muda tamanho de bloco previamente alocado: realloc.

```
1 arr = realloc( arr, 200 * sizeof( int ) );  
2 /* expande para 200 inteiros, preserva os 100  
   antigos */
```

Resumo Visual das Funções Novas do Cap. 14

Mapa de funções por categoria

Argumentos variádicos (<stdarg.h>):

Macro	Função
va_list	tipo para apontar para argumentos
va_start	inicializa va_list
va_arg	obtém próximo argumento
va_end	finaliza (sempre chamar!)

Término de programa (<stdlib.h>):

exit	encerra programa, retorna código
atexit	registra função para chamar no final
abort	término anormal (gera SIGABRT)

Alocação dinâmica (<stdlib.h>):

malloc	aloca bloco, conteúdo indefinido
calloc	aloca array, inicializa zero
realloc	redimensiona bloco
free	libera bloco

Sinais (<signal.h>):

signal	registra tratador para sinal
raise	gera sinal de dentro do programa

Arquivos:

tmpfile	cria arquivo temporário
tmpnam	gera nome único para arquivo temporário

Gabarito Rápido

Respostas Resumidas – 14.1

- a) < b) > c) » d) pipe |
 e) reticências ... f) va_start g) va_arg
 h) va_end i) argc j) argv
 k) make, Makefile l) exit m) atexit
 n) sufixo o) tmpfile p) signal
 q) raise r) calloc s) realloc